

Implementation of Dielectric ELC in ESPResSo

Bachelor's Thesis

Julien Hemminger

Simulation Technology B.Sc.

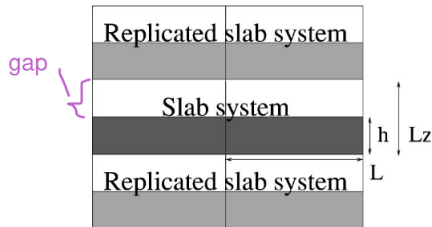
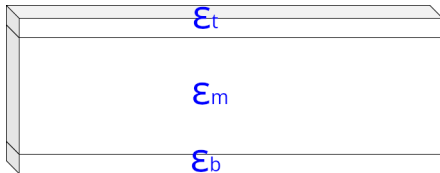
April 2026

There already is an ELC implementation that can handle dielectric interfaces in ESPReso, but:

- It has errors that are poorly understood.
- The code is unmaintainable and very difficult to read.

Goal: Replace the old ELC implementation with a clean, fixed, modern version.

Introduction



Developing in Python allows for faster iteration:

- Easier to develop, test, and prototype.
- Simplified visualization and validation of results.

Iterative/test-driven development by breaking the problem down:

- **Workflow per system:**

- Compute reference solutions (analytical or brute-force).
- Adjust ELC prototype to match reference solutions.

- **Systems analyzed:**

- Large boxes (negligible PBC influence with $L_x, L_y \gg 1$).
- Smaller, neutral boxes (no non-neutral correction term yet).
- Full parameter sweeps (Non-neutral, Madelung, random, and extreme parameters).

$$E_{2D+h} = E_{3D} + E_{\text{corr}} + E_{\text{non-neutral}} + E_{\text{dipole}}^1$$

¹Tyagi, S., Arnold, A., & Holm, C. (2008). J. Chem. Phys. 129, 204102.

Regular ELC: Results

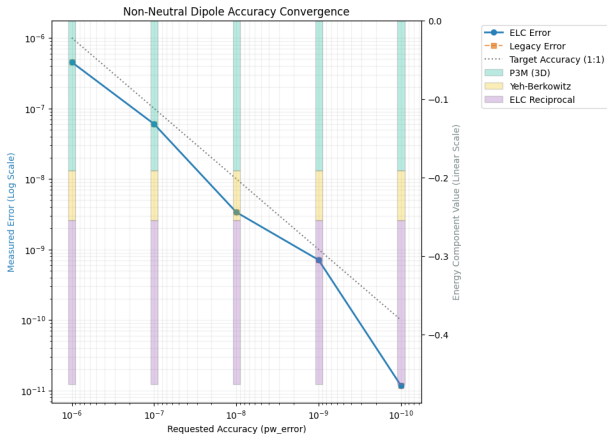
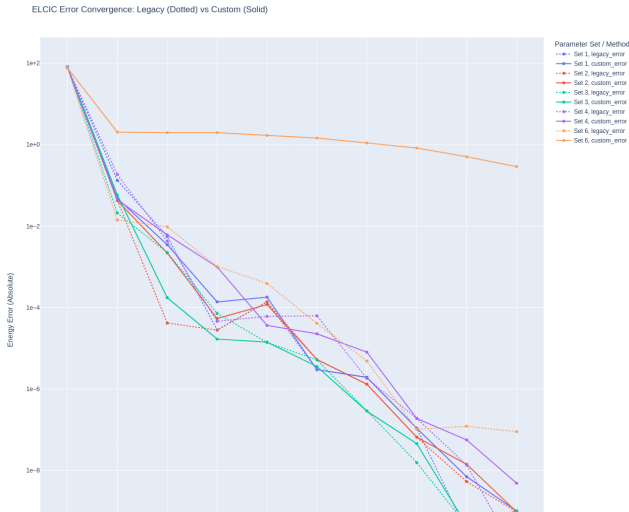


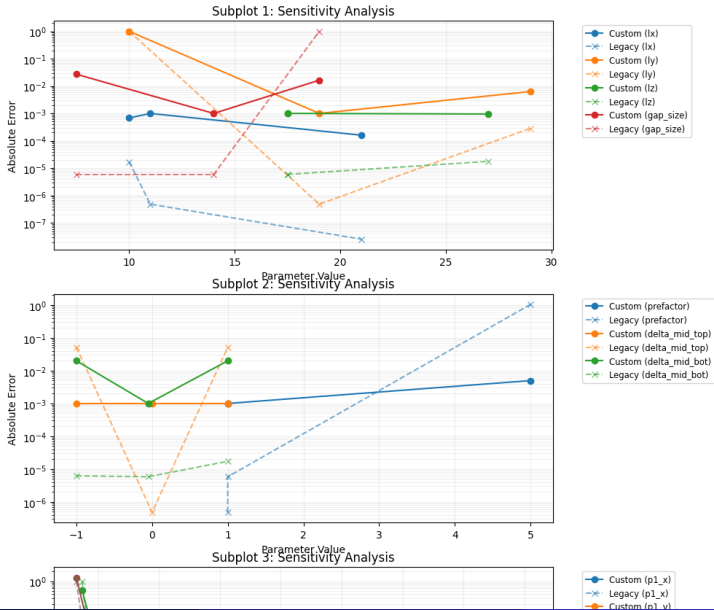
Figure: Validation of Regular ELC

Dielectric ELC: Single Plate (Current Status)

- Reference used: Ewald sum.
- Works for most parameters, though some outliers remain.

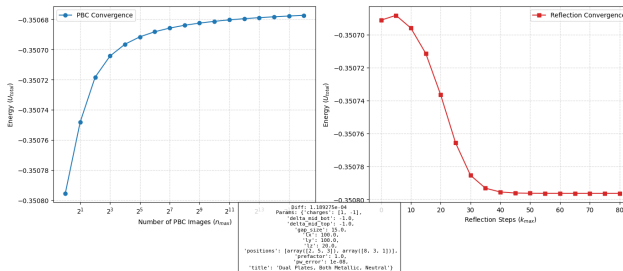


Dielectric ELC: Neighbourhood Scan



Dielectric ELC: Dual Plates

- Reference: Brute Force summation failed to converge.



Implementing in ESPResSo

- Upcoming: C++ implementation.
- Integration with **Kokkos** support for hardware acceleration.